Circuit Simulation Lab:22
Design of Counters

The purpose of this experiment is to introduce the design of Gray Counter. A gray counter changes 1-bit only during one state to another state transition. The counter is same like the normal incremental counter. The only difference is in binary representation

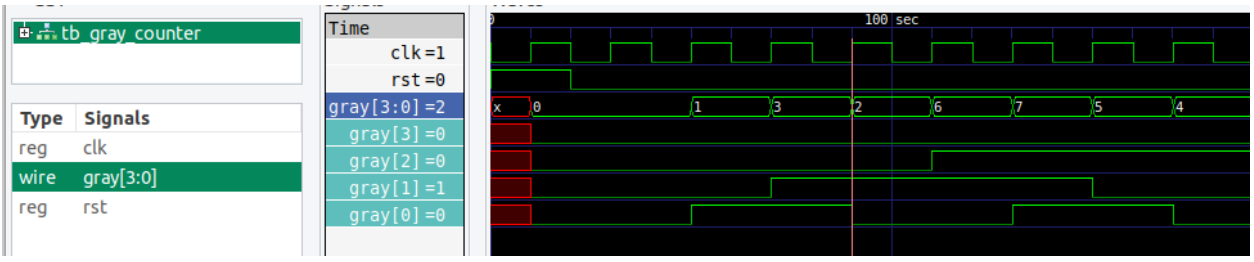
CODE:

```
module gray_counter(
input clk,
input rst,
output reg [3:0] gray
);

reg [3:0] binary;

always @(posedge clk)
begin

if(rst)
begin
   binary <= 4'd0;
   gray   <= 4'd0;
end

else
begin
   binary <= binary + 1'b1;
   gray   <= binary ^ (binary >> 1);
end

end

Endmodule
```

TESTBENCH

```
module tb_gray_counter;

reg clk;
reg rst;
```

```
wire [3:0] gray;

gray_counter dut(
.clk(clk),
.rst(rst),
.gray(gray)
);

always #10 clk = ~clk;

initial begin

$dumpfile("gray_counter.vcd");
$dumpvars(0,tb_gray_counter);

clk = 0;
rst = 1;

#20;
rst = 0;

#400;

$finish;

end

initial begin
$monitor("time=%0t gray=%b",
        $time,gray);
end

Endmodule
```



```
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_22$ iverilog -o out
gray_counter.v tb_gray_counter.v
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_22$ vvp out
VCD info: dumpfile gray_counter.vcd opened for output.
time=0 gray=xxxx
time=10 gray=0000
time=50 gray=0001
time=70 gray=0011
time=90 gray=0010
time=110 gray=0110
time=130 gray=0111
time=150 gray=0101
time=170 gray=0100
```